

The application is of four screens:

### Screen 1 - Campaign List

Fetch and display all active campaigns from the Ads API. Each campaign card must show:

- Campaign name and status badge (Active / Paused / Ended)
- Total spend vs budget with a visual progress bar
- Impressions, Clicks, and CTR (calculated client-side)
- Filter bar: All / Active / Paused
- Pull-to-refresh to re-fetch data

### Screen 2 - Campaign Detail & ML Forecast

Tapping a campaign navigates to a detail screen that:

- Fetches 30-day historical CTR data for that campaign from the Ads API
- Sends that time-series data to the ML Forecast API and displays a 7-day CTR forecast
- Renders a line chart: historical CTR (solid line) + forecast CTR (dashed line) with confidence band
- Shows a budget recommendation card based on the forecast (e.g., 'CTR predicted to rise 12% -consider increasing budget')
- Handles loading, empty, and error states explicitly with appropriate UI feedback

### Screen 3 - Spend Summary Dashboard

An aggregated summary screen showing:

- Total spend across all campaigns as a KPI card at the top
- Spend breakdown by channel (Search, Social, Display) as a pie/donut chart
- Top 3 performing campaigns by CTR, ranked in a list with percentage indicators
- Date range picker: Last 7 days / 14 days / 30 days - re-fetches data from API on change

### Screen 4 - Anomaly Alerts

A notification-style screen that:

- Polls the Ads API every 30 seconds for latest metrics
- Sends the latest metric window to the ML Anomaly Endpoint and displays returned anomaly flags
- Renders each anomaly as a color-coded alert card (red = spend spike, yellow = CTR drop)
- Shows a local push notification when a new anomaly is detected

## API Specification

**1 Ads REST API - Base URL:** <https://e5eb0d84-2b7e-4c32-98b9-233668b4e189.mock.pstmn.io/v1>

| Method | Endpoint                | Description                             | Response Shape             |
|--------|-------------------------|-----------------------------------------|----------------------------|
| GET    | /campaigns              | List all campaigns with summary metrics | { campaigns: Campaign [] } |
| GET    | /campaigns/:id          | Single campaign full detail             | { campaign: Campaign }     |
| GET    | /campaigns/:id/history  | 30-day daily CTR history                | { history: DailyMetric[] } |
| GET    | /campaigns/summary      | Aggregated spend by channel + date      | { summary: Summary }       |
| GET    | /campaigns/metrics/live | Latest metrics snapshot (for polling)   | { snapshot: Snapshot }     |

**2 ML Forecast API - Base URL:** <https://e5eb0d84-2b7e-4c32-98b9-233668b4e189.mock.pstmn.io/v1>

| Method | Endpoint        | Description                          | Response Shape              |
|--------|-----------------|--------------------------------------|-----------------------------|
| POST   | /forecast/ctr   | Send history, receive 7-day forecast | {forecast: ForecastPoint[]} |
| POST   | /anomaly/detect | Send snapshot, receive anomaly flags | {anomalies: Anomaly []}     |

### 3 Sample Request & Response Shapes

| POST /forecast/ctr - Request Body                                                                                                          | POST /forecast/ctr - Response                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>{   "campaign_id": "camp_001",   "history": [     {"date": "2025-03-01",      "ctr": 0.043},     ...   ],   "horizon_days": 7 }</pre> | <pre>{   "forecast": [     {"date": "2025-04-01",      "predicted_ctr": 0.051,      "lower_bound": 0.044,      "upper_bound": 0.059},     ...   ] }</pre> |

## Technical Requirements

### 1 Mandatory

- Flutter SDK - latest stable channel
- Dio or http package for all API calls
- Bloc or Riverpod for state management (no setState beyond widget level)
- fl\_chart or syncfusion\_flutter\_charts for all charts and graphs
- Named route navigation between all screens
- flutter\_local\_notifications for anomaly push alerts

### 2 Code Quality Expectations

- No business logic inside Widget build() methods
- All API models must be separate Dart classes with fromJson() factory constructors
- Error handling: every API call wrapped in try/catch with user-facing error messages
- Loading indicators on every async operation
- README.md with setup instructions, architecture diagram, and assumptions made

### 3 Bonus (Optional - for standout candidates)

- Offline caching of last fetched campaign list using Hive or SharedPreferences
- Animated transitions between list and detail screens
- Unit tests for at least one repository class and one bloc/provider
- Dark mode support